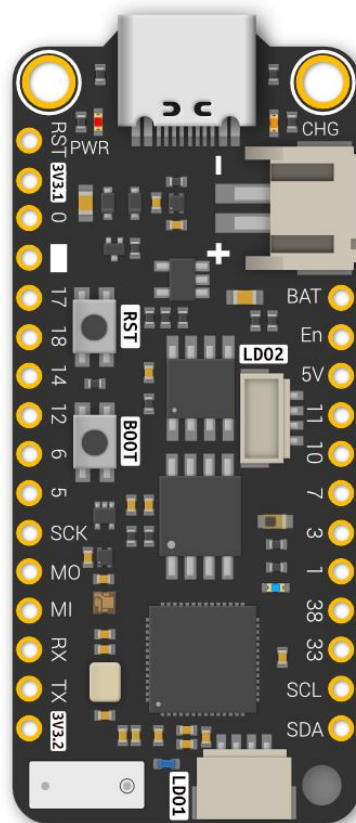


ESP32 Architecture & Embedded C/C++ Concepts

Task 2.1



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report is a research and conceptual-preparation deliverable rather than a build log: its purpose is to investigate the theoretical foundations that will be needed before starting production-grade driver development and FreeRTOS-based firmware. Four broad areas are covered. Section 2 looks inward at the ESP32 itself – its physical memory map (DRAM/IRAM/PSRAM) and the standard C/C++ memory segments a compiled program is organized into, plus two ESP32-specific keywords (`volatile` and `IRAM_ATTR`) that matter once interrupts are involved. Section 3 covers pointers, references, struct layout, and function pointers – the low-level mechanics needed to write efficient, predictable embedded C/C++. Section 4 covers the C++ OOP features (encapsulation, constructors, virtual functions, dynamic allocation) that will be used to build the production drivers, together with their concrete cost on a resource-constrained microcontroller. Section 5 closes with a short synthesis of how these concepts will directly inform upcoming driver and FreeRTOS work.

2. ESP32 Memory Architecture & Layout

2.1 The ESP32 Memory Map

The ESP32 does not have a single flat memory space. Instead, its Xtensa/RISC-V core(s) sit behind a bus matrix that exposes several physically distinct memory regions, each with a different role, capacity, and speed. The three most relevant to firmware development are summarized below.

Memory	Role	Typical Capacity	Speed / Practical Use
DRAM (Data RAM)	General-purpose on-chip RAM used for the heap, the stack, global/static variables (<code>.data/.bss</code>), and any buffer that does not need to live in instruction memory.	Classic ESP32: 520 KB total on-chip SRAM, fixed-split into 320 KB DRAM + 200 KB IRAM (of that 320 KB DRAM, only 160 KB can be statically allocated – the rest is heap-only). ESP32-S3: 512 KB of on-chip SRAM used as a single unified pool for both data and instructions, with the DRAM/IRAM split configured at link time rather than fixed in hardware.	Fast, single-cycle access from the CPU. Used for virtually all run-time data: sensor buffers, FreeRTOS task stacks, dynamically allocated objects.
IRAM (Instruction RAM)	On-chip RAM reserved for executable code that must run without the latency of fetching from external flash – most importantly, interrupt service routines (ISRs) and flash-cache-critical code.	Carved out of the same on-chip SRAM pool as DRAM: 200 KB on the classic ESP32 (fixed), and a linker-configurable region on the ESP32-S3 (since its 512 KB SRAM is not pre-split in hardware).	Same raw speed as DRAM (on-chip SRAM), but execution here is guaranteed to continue even while the external flash's instruction cache is temporarily disabled (e.g. during a flash write).
PSRAM (External SPI RAM)	Optional off-chip RAM connected over a high-speed SPI/QSPI bus, used to extend available heap for large buffers – image frames, TLS buffers, large lookup tables – that would not fit in on-chip DRAM.	Commonly 2 MB (Quad SPI) or 8 MB (Octal SPI) on modules that include it, with some modules offering up to 16 MB; the ESP32-S3 can address up to 32 MB of external PSRAM in its virtual memory space.	Noticeably slower than on-chip DRAM/IRAM due to the external SPI bus and cache-miss latency; unsuitable for ISR code or hard-real-time-critical data, but well suited for large, latency-tolerant buffers.

Table 1: ESP32 physical memory regions relevant to firmware development. Capacity figures verified against the Espressif ESP-IDF Programming Guide (Memory Types, ESP32 & ESP32-S3) and the ESP32-S3 Series Datasheet.

Interesting detail found during research: the classic ESP32 and the ESP32-S3 handle this split very differently. The classic ESP32's 320 KB/200 KB DRAM/IRAM division is a genuine hardware limitation of its SRAM banking – it is not just a linker convention. The ESP32-S3, by contrast, treats its 512 KB of SRAM as one flexible pool, letting the linker decide how much becomes IRAM based on how much `IRAM_ATTR` code the application actually contains, which is generally more RAM-efficient for applications with few IRAM-resident functions.

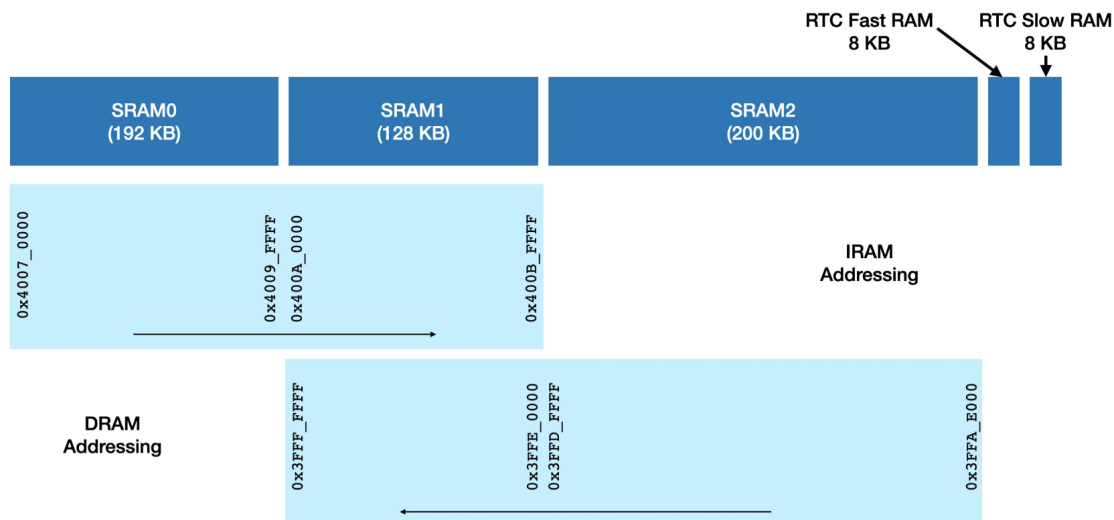


Figure 1: ESP32 memory map showing DRAM, IRAM, Flash, and PSRAM regions.

2.2 General MCU Memory Sections

Independent of the physical ESP32 memory regions above, every compiled C/C++ program (on the ESP32 or otherwise) is organized by the linker into a standard set of logical segments:

- **.text** – The compiled machine instructions (the program's executable code). Read-only at runtime.
- **.rodata** – Read-only initialized data: string literals and variables explicitly marked `const` whose value is known at compile time.
- **.data** – Initialized, writable global and static variables. Their initial values are stored in flash and copied into RAM at startup.
- **.bss** – Uninitialized (or zero-initialized) global and static variables. No space is wasted storing zeros in flash – the loader simply zero-fills this region at startup.
- **Stack** – Grows and shrinks automatically with function calls; holds local variables, function parameters, and return addresses. Fast allocation/deallocation, but limited in size (and, on FreeRTOS, each task has its own separate, fixed-size stack).
- **Heap** – Dynamically managed memory obtained via `malloc()/new` and released via `free()/delete`. Flexible in size but slower to allocate and prone to fragmentation (see Section 4.3).

Declaration	Memory Segment
<code>const int x = 5;</code> (global scope)	<code>.rodata</code>
<code>int y = 5;</code> (global, initialized)	<code>.data</code>
<code>int z;</code> (global, uninitialized)	<code>.bss</code>
<code>int k = 10;</code> (local, inside a function)	Stack
Buffer from <code>malloc()</code> or <code>new</code>	Heap

Table 2: Mapping of representative C/C++ declarations to their memory segment.

```

1 const int x = 5;          // .rodata  -- compile-time constant, read-only
2 int y = 5;              // .data    -- initialized global/static
3 int z;                  // .bss     -- uninitialized global/static (zero-filled)
4
5 void example() {
6     int k = 10;          // Stack    -- local, exists only during the call
7     int *buf = (int*)malloc(64 * sizeof(int)); // Heap -- persists until free()
8 }

```

Listing 1: Segment placement of the five example declarations

2.3 ESP32 Directives & Keywords

volatile. The `volatile` qualifier tells the compiler that a variable's value may change at any time due to something *outside* the normal flow of the program – a hardware register, or a variable written by an ISR while `main()/loop()` is reading it. Without `volatile`, the compiler is free to apply standard optimizations that are unsafe in this situation: it may cache the variable's value in a CPU register and never re-read it from memory, or it may reorder/eliminate what it thinks are "redundant" reads or writes. `volatile` disables exactly those optimizations for that variable: every access in the source code is guaranteed to become a real memory read or write, in the order written. This is why an ISR-set flag such as `volatile bool estopTriggered` must be declared `volatile` – otherwise the main loop could keep reading a stale, cached copy of the flag forever and never notice the interrupt fired.

IRAM_ATTR. This is an ESP-IDF/Arduino-ESP32 macro/attribute that forces the function it decorates to be linked into IRAM (see Section 2.1) instead of the default flash-resident code region. Normally, executable code is read from external flash through an instruction cache; this is efficient but depends on the flash's SPI bus being available. During certain operations – most notably writing to flash (e.g. NVS/OTA writes) or entering some power-management states – the ESP-IDF temporarily disables the flash cache and, at the same time, disables every interrupt that has *not* been registered as IRAM-safe, precisely to stop a non-IRAM ISR from firing while the cache is down. In pure ESP-IDF, this is the "safe" outcome: the un-flagged interrupt is simply deferred until the cache is restored, rather than firing mid-flash-write. In the Arduino-ESP32 core, however, omitting `IRAM_ATTR` on an ISR (or on any function that ISR calls) is a very common real-world crash cause – if the interrupt does still fire while the cache is disabled, the CPU faults trying to execute or read from the disabled cached region, producing the well-known **Guru Meditation Error: ... (Cache disabled but cached memory region accessed)** panic and an immediate reboot. Marking time-critical ISRs `IRAM_ATTR` guarantees their code is always resident in fast, always- available on-chip RAM, so they can run correctly and with deterministic latency regardless of what the flash subsystem is doing at that instant. A subtlety worth noting: marking a function `IRAM_ATTR` does not automatically pull in every function it *calls* – any helper function invoked from inside an IRAM ISR must independently be safe to call with the cache

disabled (itself `IRAM_ATTR`, or operating only on data already in RAM), which is a common source of the same crash even after the top-level ISR was correctly marked.

```

1 volatile bool estopTriggered = false;    // written by the ISR, read in loop()
2
3 void IRAM_ATTR onEstopChange() {
4     // Minimal, deterministic work only -- code MUST be in IRAM,
5     // since this can fire while the flash cache is disabled.
6     estopTriggered = true;
7 }

```

Listing 2: volatile and `IRAM_ATTR` used together for a debounced E-Stop ISR

3. Pointers, References & Memory Management

3.1 Pointers vs. References (C vs. C++)

Property	Pointer (*)	Reference (&)
Nullability	Can be null (<code>nullptr</code>); must be checked before dereferencing.	Cannot be null – a reference must be bound to a valid object at the point it is declared.
Reassignment	Can be reassigned to point at a different object at any time.	Cannot be re-bound after initialization; it is an alias for the <i>same</i> object for its entire lifetime.
Pointer Arithmetic	Supported (<code>ptr++</code> , <code>ptr + n</code>) – useful for arrays/buffers, but a source of bugs if misused.	Not supported; a reference has no notion of “the next reference.”
Syntax	Explicit dereference required: <code>*ptr</code> to access the value, <code>ptr->member</code> for members.	Used exactly like the original variable – no dereference operator needed at the call site.

Table 3: Pointers vs. references.

3.2 Performance: Passing by const Reference

For a large struct such as a sensor-reading bundle, three passing strategies are possible, and they are not equivalent:

- **By value** (`SensorData data`) copies the *entire* struct onto the stack for every function call – wasted CPU cycles and stack space proportional to the struct’s size, which matters directly on a memory-constrained MCU.
- **By raw pointer** (`SensorData *data`) avoids the copy, but costs safety: the pointer could be null, and nothing in the signature stops the function from mutating the caller’s data or reassigning the pointer to point elsewhere.

- **By const reference** (`const SensorData &data`) gets the best of both: internally a reference is typically implemented as a pointer, so passing one costs only a single machine word (no struct copy), while the `const` qualifier lets the compiler *enforce*, at compile time, that the function cannot modify the caller's data. It is also guaranteed non-null (assuming the caller passed a valid object), so no defensive null-check is required inside the function.

This is why `const SensorData &data` is the preferred embedded C++ idiom for read-only access to a non-trivial struct: it has pointer-level performance with value-semantics safety.

3.3 Structs, Enums, and Unpacking Data

C-style enum vs. C++11 enum class. A plain C-style `enum` injects its enumerator names directly into the surrounding scope and implicitly converts to `int`, which means two unrelated enums can silently collide (`RUNNING` from a motor-state enum and `RUNNING` from an unrelated enum cannot coexist) and an enum value can be compared or assigned across completely unrelated types without a compiler warning. A `enum class` (scoped enum) fixes both problems: its enumerators are scoped to the enum's own name (`MotorState::RUNNING`), and it does *not* implicitly convert to `int` or to any other enum type. For a state machine, this matters directly – accidentally comparing `MotorState` against an unrelated `LedState` value, or letting an arbitrary integer silently masquerade as a valid state, becomes a compile-time error instead of a latent runtime bug.

```

1 enum LegacyState { STOPPED, RUNNING };           // pollutes global scope,
2                                                    // implicitly converts to int
3
4 enum class MotorState { STOPPED, RUNNING };      // scoped + type-safe
5 MotorState s = MotorState::RUNNING;              // must be explicitly qualified
6 // if (s == 1) { }                               // compile error -- no implicit int
   conversion

```

Listing 3: C-style enum vs. enum class

Struct Padding & Alignment. The compiler does not necessarily pack a struct's members back-to-back in memory. Each member type has an alignment requirement (an `int` typically must start at an address divisible by 4, a `double` by 8, etc.), and the compiler inserts invisible padding bytes between members – and sometimes after the last member – so that every member, and the struct as a whole when placed in an array, satisfies its alignment requirement. This directly affects RAM usage: a struct with poorly ordered members (e.g. alternating `char` and `int`) can end up significantly larger than the sum of its members' sizes, simply due to padding. The `__attribute__((packed))` directive (GCC/Clang) instructs the compiler to omit this padding entirely, forcing members to sit at the smallest possible offsets. This saves RAM and is essential when a struct's layout must exactly match an external format (e.g. a sensor's raw register map or a wire/network protocol), but it comes at a cost: unaligned access on some architectures is slower, and on others it is outright illegal and faults – so `packed` should be used deliberately, not by default.

```

1 struct Unpacked {                               // sizeof == 8 on a typical 32-bit target
2     char  flag;                                  // 1 byte + 3 bytes padding
3     int   value;                                 // 4 bytes
4 };
5
6 struct __attribute__((packed)) Packed {         // sizeof == 5, no padding
7     char  flag;
8     int   value;
9 };

```

Listing 4: Struct padding vs. a packed struct

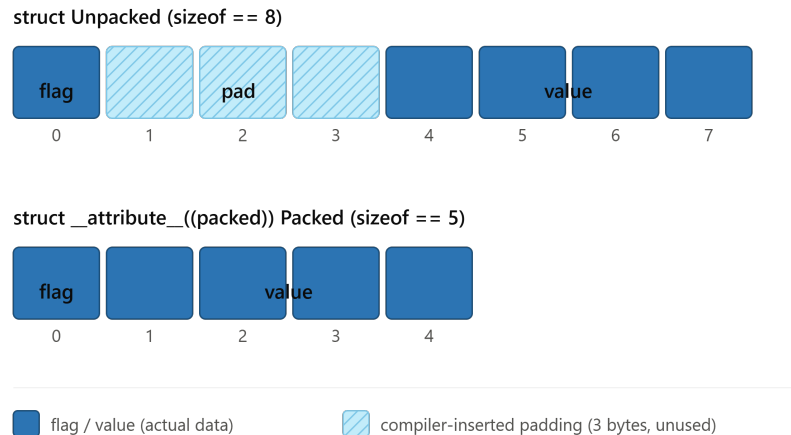


Figure 2: Struct member layout with padding bytes highlighted (Unpacked vs. Packed).

3.4 Function Pointers & Callbacks

A function pointer is a variable that stores the address of a function rather than the address of a data object, and can be called through just like the function itself. Because it is a value, a function pointer can be stored in a struct, passed as a parameter, or reassigned at runtime – which is exactly what an event-driven callback needs. Instead of hard-coding what happens when a button is pressed or a timer expires, a driver or interrupt subsystem is written once to simply “call whatever function pointer was registered,” and different application code can register different handlers without the low-level driver needing to know anything about them in advance. This is the mechanism underlying `attachInterrupt(pin, callbackFn, mode)` in the Arduino/ESP32 framework: the interrupt hardware fires, the runtime looks up the function pointer that was registered for that pin, and calls it.

```

1 typedef void (*CallbackFn)();           // type alias for "pointer to void()"
2
3 void onButtonPress() { /* handle event */ }
4
5 CallbackFn registeredCallback = onButtonPress; // store the function's address
6
7 void triggerEvent() {
8     if (registeredCallback) registeredCallback(); // invoke through the pointer
9 }
10
11 // Real-world equivalent:
12 // attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), onButtonPress, FALLING);

```

Listing 5: A function pointer used as an interrupt/timer callback

4. C++ OOP & Access Modifiers in Embedded Systems

4.1 Access Specifiers (public, private, protected)

Encapsulation is the practice of bundling an object's internal data together with the functions that operate on it, and restricting direct access to that data from outside the object – external code may only interact with it through a deliberately designed public interface. In an embedded driver class, hardware pin numbers and internal sensor-calibration values should be **private** because they represent *internal implementation state* that, if modified directly and arbitrarily from outside the class, can put the object into an invalid or unsafe configuration – e.g. changing a motor driver's PWM pin after `begin()` has already configured the hardware timer for the old pin, or overwriting a sensor's calibration offset with an unvalidated value. Keeping these fields private and exposing only vetted public methods (`forward(speed)`, `getCalibratedZero()`, ...) means every state change is forced through code that can validate it, guaranteeing the object's internal invariants always hold.

private vs. **protected** matters specifically when a class is **inherited**. A **private** member of a base class is accessible only inside the base class itself – not even a derived class can touch it directly. A **protected** member is accessible inside the base class *and* inside any class that derives from it, while remaining inaccessible from outside the class hierarchy entirely. In practice, a base driver class exposes its low-level state as **protected** when it expects derived classes to need direct access to it (e.g. a base `MotorDriver` class exposing a **protected** `int _speed` that a derived `PIDMotorDriver` needs to read/adjust directly), and keeps it **private** when even derived classes should only interact with that state through the base class's own methods.

4.2 Constructors & Initialization Lists

A constructor is a special member function, automatically invoked when an object is created, whose job is to bring the object into a valid initial state – setting up pin modes, default values, or acquiring any resource the object needs. A destructor is invoked automatically when the object is destroyed, and is responsible for releasing whatever the constructor acquired (closing a peripheral, freeing dynamically allocated memory), ensuring no resource is leaked when the object goes out of scope.

Member Initializer Lists should be preferred over assigning member variables inside the constructor body for both correctness and performance reasons. When a member is assigned inside the body, C++ first *default-constructs* that member (using its own default constructor, or zero-initializing a built-in type) and only *then* assigns the desired value on the next line – effectively initializing it twice. A member initializer list constructs each member directly with its intended value in a single step, skipping the redundant default construction entirely. This distinction is not just a style preference for members that are **const** or references: those *must* be given a value in the initializer list, because they cannot be assigned to at all once the constructor body begins executing.

```

1  class L298N {
2  public:
3      // Less efficient: each member is default-constructed, THEN assigned
4      L298N(int in1, int in2, int ena) {
5          _in1 = in1; _in2 = in2; _ena = ena;
6      }
7
8      // Preferred: each member constructed once, directly with its value
9      L298N(int in1, int in2, int ena) : _in1(in1), _in2(in2), _ena(ena) {}
10
11 private:
12     int _in1, _in2, _ena;
13 };

```

Listing 6: Assignment in the body vs. a member initializer list

4.3 Cost of C++ Features in Embedded Systems

Virtual functions and the vtable. A class with one or more **virtual** functions gains a hidden pointer member (the vtable pointer), added by the compiler to every instance of that class, which points to a per-class table of function-pointer entries (the virtual table / vtable) used to resolve which override to call at runtime. This has two costs: a small memory cost (one extra pointer per object, plus one static table per class – typically only a few bytes to tens of bytes total), and a small but real CPU cost per call, since a virtual call requires an extra indirection (load the vtable pointer, then load the function pointer from the table, then call it) compared to a direct, statically-resolved function call. On a microcontroller this is rarely prohibitive for occasional calls, but it does prevent the compiler from inlining the call, and in a tight, high-frequency loop (e.g. a control loop running every PWM cycle) the extra indirection and lost inlining opportunity can be measurable – which is why performance-critical embedded code sometimes deliberately avoids virtual dispatch in favor of templates or direct calls. Concretely, on the ESP32 – whose CPU cores (Xtensa LX6/LX7, or RISC-V on newer variants) are all 32-bit architectures – a pointer is 4 bytes, so the hidden vtable pointer adds exactly 4 bytes to every instance of a class with at least one virtual function; for a small sensor/driver object that might otherwise be only a handful of bytes, that 4-byte pointer can be a genuinely non-trivial fraction of the object's total size when many instances exist.

Dynamic memory allocation (**new/delete/malloc/free**). On a microcontroller, the heap and the stack(s) typically share a fixed, comparatively small pool of RAM (often further subdivided per-task under FreeRTOS). Two problems make dynamic allocation risky in this environment: fragmentation and non-determinism. Repeated alloc/free cycles of varying sizes over a long-running program (an embedded device may run for months without a reboot) can leave the heap fragmented into many small free blocks – enough total free memory may exist, yet no single block may be large enough to satisfy a new allocation request, causing **malloc()**/**new** to fail unpredictably long after the system appeared to be working fine. Separately, the *time* a heap allocator takes to find a suitable free block is generally not constant – it can vary run to run depending on the current fragmentation state – which conflicts directly with the deterministic, bounded-latency requirements of real-time embedded code (especially inside or near an ISR, where dynamic allocation should essentially never be used). The stack, by contrast, allocates and frees in strict last-in-first-out order in constant time, with no fragmentation possible, which is why embedded style guides generally prefer stack allocation, fixed-size static buffers, or allocate-once-at-startup patterns over allocating/freeing on the heap during normal runtime operation.

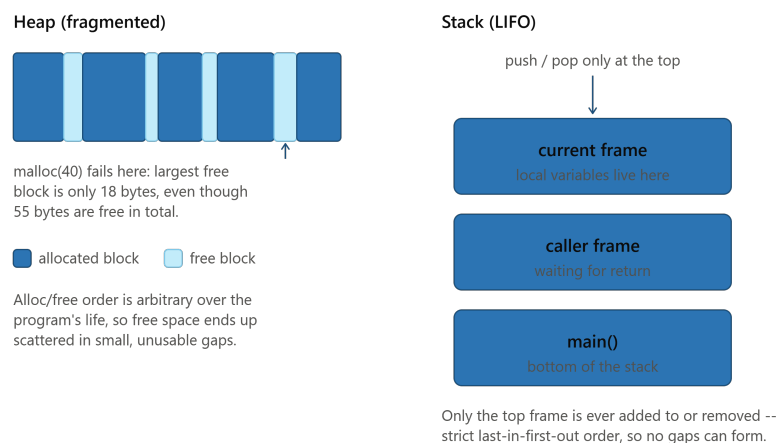


Figure 3: Fragmented heap (scattered free blocks) vs. a LIFO stack.

5. Summary & Next Steps

The concepts investigated in this report form the theoretical groundwork for the production driver work and FreeRTOS integration that follow. The ESP32 memory map (Section 2.1) and `IRAM_ATTR/volatile` (Section 2.3) are directly relevant to writing correct, fast ISRs – something FreeRTOS-based firmware relies on heavily for timers, GPIO events, and inter-task signaling. The pointer/reference and struct-layout material (Section 3) will shape how sensor data is passed between driver objects and tasks without unnecessary copying or wasted RAM. Finally, the OOP cost analysis (Section 4.3) – particularly the caution around virtual dispatch in hot loops and dynamic allocation on a long-running, fragmentation-sensitive heap – will directly inform the design decisions for the upcoming production drivers: preferring stack/static allocation, using `const` references for passing sensor structs, and reserving virtual functions for genuinely polymorphic, non-time-critical interfaces rather than every class in the system.

6. References

1. Espressif Systems, *ESP-IDF Programming Guide – Memory Types* (ESP32 and ESP32-S3 targets). <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/memory-types.html>
2. Espressif Systems, *ESP32-S3 Series Datasheet*, v2.2. https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
3. Espressif Systems, *ESP-IDF Programming Guide – Support for External RAM (ESP32-S3)*. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/external-ram.html>
4. Espressif Developer Portal, *ESP32 Programmers' Memory Model*. <https://developer.espressif.com/blog/esp32-programmers-memory-model/>
5. Espressif, *ESP-IDF Programming Guide – Fatal Errors* (Guru Meditation / cache-disabled panic causes). <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/fatal-errors.html>
6. `espressif/arduino-esp32` GitHub issue tracker – real-world `IRAM_ATTR`-omission crash reports ("*Cache disabled but cached memory region accessed*"), used to verify the practical consequence described in Section 2.3.
7. Random Nerd Tutorials / Last Minute Engineers, ESP32 and ESP32-S3 GPIO/ADC pinout references, used to cross-check ADC1/ADC2 channel assignments.
8. ISO/IEC 14882 (C++ Standard) – general semantics of references, `const`, scoped enumerations, and virtual dispatch; Stroustrup, *The C++ Programming Language*, for the standard rationale behind member initializer lists and encapsulation.